

SIMATS ENGINEERING ASSESSMENT TOOL2

COURSE CODE: CSA14

COURSE NAME: Compiler Design

COURSE OUTCOME COVERED

CO4: Examine intermediate code generation techniques to enhance code optimization (BL4)

Assessment Tool Weightage: 20%

QUESTIONS: 5

Total Marks:50

Annexure A

INDUSTRY PROBLEM- SOLVING TASK

Code of

V.Aravind Chowdary/192321119(Name / Reg No) certify that this submission is work and that I have adhered to the guidelines specified for Iunderstand that any violation of academic integrity rules will result in

Signature of the M.V.Rakesh Reddy

Q.	Problem Understanding (20)	Method Selection (20)	Solution Process (25)	Accuracy Calculation (20)	Interpretation of (15)	Total Score (out 100)
1	/ 4	/	/ 4	/ 4	/ 4	
2	/ 4	4	/ 4	/ 4	/ 4	
3	/ 4	/	/ 4	/ 4	/ 4	
4	/ 4	4	/ 4	/ 4	/ 4	
5	/ 4	/	/ 4	/ 4	/ 4	
OVERALL RUBRICS VALUE						
	/ 4	44	/ 4	/ 4	/ 4	
	/ 25	/ 25	/ 20	/ 20	/ 10	/ 100

Annexure A

INDUSTRY PROBLEM- SOLVING TASK

1. Banking Transaction Processing System

A developing a compiler for a banking software company is domain-specific scripting language used in transaction processing systems. The compiler must translate variable declarations, assignment statements, and Boolean expressions efficiently to avoid transaction failures during peak hours. The system also uses case statements to classify transactions such as deposit, withdrawal, and loan processing. During testing, incorrect intermediate code generation caused delays in transaction execution.

Tasks:

- (i) Design suitable intermediate code for the given assignment and Boolean expressions. (K3)
- (ii) Explain how backpatching can help in handling conditional transaction validation. (K4)
- (iii) Analyze the role of intermediate languages in improving banking software portability. (K5)

2. Building a smart traffic management. Smart Traffic Control System

An application where signals are controlled automatically based on traffic density. An automotive company is building a smart traffic management system. The compiler used in the embedded controller must process Boolean expressions and case statements for different traffic conditions. Due to inefficient intermediate code generation, the response time of traffic signals increased significantly during rush hours. The development team plans to optimize procedure calls and assignment statements in the compiler design.

Tasks:

- (i) Construct intermediate code for the traffic signal control logic. (K3)
- (ii) Explain how procedure calls improve modularity in the traffic control application. (K4)
- (iii) Evaluate the importance of compiler optimization in real-time embedded systems. (K5)

3. E-Commerce Product Recommendation Engine

An e-commerce company uses a rule-based recommendation engine developed using a custom programming language. The compiler processes declarations, assignment statements, and Boolean conditions to generate recommendations dynamically. During seasonal sales, delays occurred because of improper handling of case statements and inefficient intermediate code generation. The organization wants to redesign the compiler to improve execution speed and maintainability.

Tasks:

- (i) Generate intermediate representation for the recommendation logic. (K3)
- (ii) Discuss how case statements can efficiently handle product category selection. (K4)
- (iii) Analyze the role of compiler design in improving large-scale ecommerce applications. (K5)

4. Hospital Patient Monitoring System

A healthcare technology company is developing a patient monitoring system that continuously checks vital signs using embedded software. The compiler used in the system must efficiently evaluate Boolean expressions and assignment statements for emergency alerts. Improper backpatching in conditional branching caused delayed alarm generation during critical patient conditions. The software team is redesigning the compiler for faster and more reliable intermediate code execution.

Tasks:

- (i) Design intermediate code for patient condition monitoring logic. (K3)
- (ii) Explain the significance of backpatching in emergency alert systems. (K4)
- (iii) Evaluate how compiler optimization improves reliability in healthcare applications. (K5)

5. Online Airline Reservation System

An airline company is modernizing its reservation platform using a custom scripting language for ticket booking and seat allocation. The compiler must handle declarations, assignment statements, and procedure calls efficiently to manage thousands of booking requests simultaneously. During testing, errors in intermediate code generation affected seat allocation and payment confirmation processes. The software architects aim to improve compiler performance for large-scale distributed applications.

Tasks:

- (i) Develop intermediate code for ticket booking and seat allocation operations. (K3)

- (ii) Explain how procedure calls support modular reservation management. (K4)
- (iii) Analyze the role of intermediate languages in distributed airlinereservation systems. (K5)

Set 2

1. Industrial Robot Automation System

A manufacturing company is developing an industrial robot control system where robotic arms perform assembly operations automatically. The compiler used in the embedded software processes declarations, assignment statements, and Boolean expressions to coordinate robotic movement and safety checks. During production, delays occurred due to inefficient handling of procedure calls and intermediate code generation. The company plans to redesign the compiler to improve execution speed and synchronization between robotic units. Tasks:

- (i) Design intermediate code for robotic movement and safety validationlogic. (K3)
- (ii) Explain how procedure calls improve modularity in industrialautomation systems. (K4)
- (iii) Analyze the role of compiler optimization in improving manufacturingefficiency. (K5)

2. Cybersecurity Intrusion Detection System

A cybersecurity firm is developing an intrusion detection system that continuously monitors network traffic for suspicious activities. The compiler processes Boolean expressions, assignment statements, and case statements to identify different categories of cyberattacks. During real-time monitoring, inefficient intermediate code generation increased system response time and delayed threat detection. The development team intends to enhance compiler efficiency for faster and more reliable security analysis. Tasks:

- (i) Generate intermediate representation for network threat detectionlogic. (K3)
- (ii) Discuss how case statements can efficiently classify multiplecyberattack scenarios. (K4)
- (iii) Evaluate the importance of compiler optimization in real-timecybersecurity applications. (K5)

3. Digital Payment Gateway System

A fintech company is building a digital payment gateway to process online transactions securely and efficiently. The compiler used in the transaction engine must evaluate Boolean conditions, assignment statements, and procedure calls during payment verification and fraud detection. During peak transaction periods, improper backpatching caused delays in payment confirmation and rollback operations. The organization plans to redesign the compiler to improve transaction reliability and scalability.

Tasks:

- (i) Construct intermediate code for payment verification and fraud detection logic. (K3)
- (ii) Explain how backpatching supports conditional transaction processing. (K4)
- (iii) Analyze the role of compiler design in ensuring secure and scalable fintech applications. (K5)

4. Smart Agriculture Monitoring System

An agritech company is developing a smart agriculture platform that monitors soil moisture, temperature, and irrigation systems automatically. The compiler in the embedded controller processes declarations, assignment statements, and Boolean expressions to activate irrigation based on environmental conditions. Inefficient handling of intermediate code caused delays in irrigation control during critical farming conditions. The software team aims to improve compiler performance for real-time agricultural automation. Tasks:

- (i) Develop intermediate code for irrigation control and environmental monitoring. (K3)
- (ii) Explain the role of Boolean expressions in automated farming systems. (K4)
- (iii) Evaluate how compiler optimization enhances efficiency in smart agriculture applications. (K5)

5. Cloud-Based Video Streaming Platform

A multimedia company is developing a cloud-based video streaming platform that dynamically adjusts video quality based on network conditions. The compiler used in the streaming engine processes case statements, assignment statements, and procedure calls for resource

allocation and bandwidth management. During high user traffic, inefficient intermediate code generation caused buffering delays and reduced streaming quality. The company plans to redesign the compiler to improve scalability and performance. Tasks:

(i) Design intermediate code for bandwidth allocation and quality adaptation logic. (K3)

(ii) Explain how case statements help manage multiple streaming quality levels. (K4)

(iii) Analyze the significance of compiler optimization in cloud-based multimedia systems. (K5)

Set 3

1. Autonomous Drone Delivery System

A logistics company is developing an autonomous drone delivery system to transport packages across urban areas. The compiler embedded in the drone controller processes declarations, assignment statements, and Boolean expressions to manage navigation, obstacle detection, and battery monitoring. During testing, inefficient intermediate code generation caused delays in route adjustment and emergency landing procedures. The organization plans to optimize the compiler for reliable real-time drone operations.

Tasks:

- (i) Design intermediate code for drone navigation and obstacle detection logic. (K3)
- (ii) Explain how Boolean expressions support autonomous decision-making in drones. (K4)
- (iii) Analyze the importance of compiler optimization in real-time delivery systems. (K5)

2. Railway Reservation and Scheduling System

A railway management company is modernizing its reservation and scheduling platform using a custom programming language. The case compiler processes declarations, assignment statements, statements, and procedure calls for seat allocation, train scheduling, and passenger status updates. During peak booking hours, improper intermediate code generation caused delays in reservation confirmation and ticket cancellation. The development team aims to redesign the compiler to improve scalability and transaction efficiency.

Tasks:

- (i) Construct intermediate code for reservation and scheduling operations. (K3)
- (ii) Discuss how case statements efficiently handle passenger ticket categories. (K4)
- (iii) Evaluate the role of compiler optimization in large-scale transportation systems. (K5)

3. Intelligent Energy Management System

An energy company is building a smart grid monitoring system that automatically power controls electricity distribution based on consumption. The compiler in the embedded controller evaluates Boolean expressions and assignment statements to monitor load balancing and fault detection. During high-demand periods, inefficient backpatching delayed fault recovery and power redistribution. The company plans to enhance compiler performance to improve reliability and energy efficiency.

Tasks:

- (i) Generate intermediate code for load balancing and fault detection logic. (K3)
- (ii) Explain the significance of backpatching in smart energy management systems. (K4)
- (iii) Analyze how compiler optimization improves the performance of smart grids. (K5)

4. AI-Based Fraud Detection System

A financial institution is developing an AI-based fraud detection platform that continuously analyzes transaction patterns. The compiler processes declarations, assignment statements, and Boolean conditions to identify suspicious activities and trigger alerts. During large-scale transaction processing, inefficient procedure calls increased response time and delayed fraud notifications. The software architects plan to redesign the compiler for faster analysis and improved scalability.

Tasks:

- (i) Develop intermediate code for fraud detection and alert generation logic. (K3)
- (ii) Explain how procedure calls improve modularity in fraud detection systems. (K4)
- (iii) Evaluate the role of compiler design in secure financial applications. (K5)

5. Smart Warehouse Inventory Management System

A retail company is implementing a smart warehouse inventory system that automatically tracks product movement and stock levels. The compiler processes assignment statements, Boolean expressions, and case statements to update inventory records and manage automated storage systems. During seasonal sales, delays in intermediate code execution affected stock synchronization and order fulfillment. The company plans to optimize the compiler to improve inventory accuracy and warehouse efficiency.

Tasks:

- (i) Design intermediate code for inventory tracking and stock update operations. (K3)
- (ii) Explain how case statements help manage multiple product categories efficiently. (K4)
- (iii) Analyze the importance of compiler optimization in warehouse automation systems. (K5)

Set4

1. Smart Home Automation System

A home automation company is developing an intelligent control system that automatically manages lighting, temperature, and security devices. The compiler embedded in the controller processes declarations, assignment statements, and Boolean expressions to activate devices based on sensor inputs and user preferences. During real-time execution, inefficient intermediate code generation caused delays in

device response and security alerts. The development team plans to redesign the compiler to improve automation efficiency and reliability.

Tasks:

- (i) Design intermediate code for smart device control and sensor monitoring logic. (K3)
- (ii) Explain how Boolean expressions support automated decision-making in smart homes. (K4)
- (iii) Analyze the role of compiler optimization in improving home automation systems. (K5)

2. Online Examination Management System

An educational technology company is building an online examination platform that evaluates students automatically and manages exam scheduling. The compiler processes declarations, assignment statements, case statements, and procedure calls for student

authentication online and result generation. During large-scale examinations, inefficient intermediate code execution caused delays in question loading and score processing. The organization plans to optimize the compiler to improve scalability and performance.

Tasks:

- (i) Construct intermediate code for student authentication and evaluation logic. (K3)
- (ii) Discuss how case statements help manage different examination categories. (K4)
- (iii) Evaluate the importance of compiler optimization in large-scale online examination systems. (K5)

3. Intelligent Weather Forecasting System

A meteorological department is developing an intelligent weather forecasting application that analyzes environmental data in real time. The compiler processes assignment statements and Boolean expressions to monitor temperature, humidity, and rainfall conditions. During severe weather conditions, improper backpatching delayed alert generation and emergency notifications. The software engineers aim to redesign the compiler to improve prediction accuracy and system responsiveness.

Tasks:

- (i) Generate intermediate code for weather monitoring and alert generation logic. (K3)
- (ii) Explain the role of backpatching in handling emergency weather alerts. (K4)

- (iii) Analyze how compiler optimization enhances real-time forecasting applications. (K5)

4. Automated Library Management System

A university is implementing an automated library management system to handle book issuance, returns, and digital catalog management. The compiler processes declarations, assignment statements, and procedure calls for member verification and book tracking operations. During semester examinations, inefficient intermediate code generation delayed transaction updates and fine calculations. The university plans to improve compiler efficiency for faster and more reliable library services.

Tasks:

- (i) Develop intermediate code for book issue and return operations. (K3)
- (ii) Explain how procedure calls improve modularity in library management systems. (K4)
- (iii) Evaluate the significance of compiler optimization in educational information systems. (K5)

5. Industrial Water Quality Monitoring System

An environmental engineering company is developing a water quality monitoring system that continuously checks pH levels, contamination, and chemical composition in industrial plants. The compiler in the embedded controller evaluates Boolean expressions, assignment statements, and case statements for automated monitoring and alert generation. During critical contamination events, inefficient intermediate code execution delayed warning notifications and corrective actions. The company aims to redesign the compiler to improve reliability and response time.

Tasks:

- (i) Design intermediate code for contamination detection and alert handling logic. (K3)
- (ii) Explain how case statements efficiently manage different water quality conditions. (K4)
- (iii) Analyze the importance of compiler optimization in industrial environmental monitoring systems. (K5)

Set 5

1. Smart Parking Management System

A smart city development company is designing an automated parking management system to monitor vehicle entry, slot allocation, and parking

fee calculation. The compiler embedded in the control software processes declarations, assignment statements, Boolean expressions, and case statements to manage parking availability and security verification. During peak hours, inefficient intermediate code generation caused delays in slot assignment and payment confirmation. The company plans to redesign the compiler to improve real-time response and system efficiency.

Tasks:

- (i) Design intermediate code for parking slot allocation and fee calculation logic. (K3)
- (ii) Explain how Boolean expressions and case statements help manage parking operations efficiently. (K4)
- (iii) Analyze the role of compiler optimization in improving smart parking management systems. (K5)

2. Digital Healthcare Appointment System

A healthcare organization is developing a digital appointment management platform to schedule doctor consultations and manage patient records. The compiler processes declarations, assignment statements, and procedure calls to handle appointment booking, cancellation, and patient verification. During high patient traffic, inefficient intermediate code execution caused delays in appointment confirmation and report generation. The development team plans to optimize the compiler for faster and more reliable healthcare services.

Tasks:

- (i) Construct intermediate code for appointment scheduling and patient verification logic. (K3)
- (ii) Explain how procedure calls improve modularity in healthcare management systems. (K4)
- (iii) Evaluate the importance of compiler optimization in digital healthcare applications. (K5)

3. Automated Toll Collection System

A transportation authority is implementing an automated toll collection system that identifies vehicles and processes payments electronically. The compiler embedded in the toll management software evaluates Boolean vehicle expressions and assignment statements for classification and transaction validation. During festival seasons, improper backpatching caused delays in payment verification and traffic management. The organization aims to redesign the compiler to improve transaction speed and operational efficiency.

Tasks:

RUBRICS FOR CALCULATION

Numerical Problems					
Criteria	Weightage	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)
Problem Understanding	20%	Clearly interprets problem, identifies all parameters and requirements accurately	Understands problem with minor gaps	Partial understanding; misses some key elements	Misinterprets or unable to understand problem
Method Selection	20%	Selects most appropriate method/with strong justification	Selects correct method with minor errors	Inappropriate or partially correct method	Unable to select method
Solution Process	25%	Logical, systematic steps with correct approach throughout	Mostly correct steps with minor mistakes	Disorganized steps; multiple errors	No clear process
Accuracy of Calculation	20%	All calculations accurate; correct final answer	Minor calculation errors	Frequent errors; incorrect final answer	No valid calculations
Interpretation of Results	15%	Clearly interprets results with units and engineering relevance	Basic interpretation with minor omissions	Limited or unclear interpretation	No interpretation

Set 1

1. Banking Transaction Processing System

i) **Intermediate code** For example, let the transaction input be `amount`, `balance`, and `type`, and let validation be `amount > 0 && amount <= balance`. A suitable three-address representation is: t1 = amount > 0 t2 = amount <= balance

t3 = t1 AND t2 ifFalse

t3 goto L_error t4 =

balance - amount

balance = t4 goto

L_end

L_error: call reportInvalidTransaction

L_end:

For case classification:

if type == 1 goto L_deposit

if type == 2 goto

L_withdraw if type == 3

goto L_loan goto L_invalid

L_deposit: call processDeposit; goto L_endcase

L_withdraw: call processWithdrawal; goto L_endcase

L_loan: call processLoan; goto L_endcase

L_invalid: call reportInvalidType

L_endcase:

One-line explanation: Three-address code separates simple operations into temporaries and labels, making control flow explicit. ii) **Backpatching** For a Boolean condition such as `amount > 0 && amount <= balance`, the compiler creates

conditional jumps whose target labels may not yet be known. The true and false jump lists are stored and later filled using backpatching when the destination labels are created. For transaction validation, a true branch can lead to authorization/processing and a false branch to rejection or rollback.

One-line explanation: Backpatching fills unresolved jump targets after the required labels become known, which simplifies generation of forward conditional branches.

iii) **Intermediate languages and portability** An intermediate language such as three-address code, SSA, or bytecode provides a machine-independent representation between source code and target machine code. The same front end can translate the banking language into the intermediate form, while different back ends generate code for different processors or platforms. Optimization can also be performed on the intermediate representation before target-specific code generation.

One-line explanation: A machine-independent intermediate representation lets one compiler front end support multiple target platforms with reusable optimization passes.

2. Smart Traffic Control System

i) **Intermediate code** Example for density-based signal control:

t1 = density > HIGH

if t1 goto L_high t2 =

density > MEDIUM if

t2 goto L_medium

goto L_low

L_high: call setRedLong; call setGreenMain; goto L_end

L_medium: call setRedMedium; call setGreenMain; goto

L_end L_low: call setRedShort; call setGreenMain L_end:

For a Boolean safety condition:

t3 = pedestrian == 0 t4 =

emergency == 0 t5 = t3

AND t4 ifFalse t5 goto

L_safe_stop

call allowTraffic

goto L_done

L_safe_stop: call stopTraffic

L_done:

One-line explanation: Labels and temporaries make real-time decisions explicit and allow later optimization. **ii) Procedure calls** Operations such as `setRedLong`, `setGreenMain`, `stopTraffic`, and `readSensor` can be implemented as separate procedures. Each procedure encapsulates one function and can be called from several traffic-control paths. This avoids duplicating code and makes testing, maintenance and replacement of individual control functions easier.

One-line explanation: Procedure calls divide the control system into reusable modules, improving maintainability and reducing duplication.

iii) Compiler optimization In a real-time embedded controller, optimization reduces unnecessary instructions, memory accesses and branches. Constant propagation, dead-code elimination, common-subexpression elimination, strength reduction, efficient procedure-call handling and register allocation can reduce execution time. The compiler should preserve deterministic timing because delayed signal decisions can reduce traffic-control quality.

One-line explanation: Optimization is important because lower execution time and predictable control flow improve the responsiveness of embedded traffic systems.

3. E-Commerce Product Recommendation Engine

i) Intermediate representation Let `category`, `price`, `stock`, and `customerScore` determine a recommendation:

```
t1 = stock > 0
t2 = price < limit
t3 = customerScore >= threshold
t4 = t1 AND t2 t5 = t4 AND t3
iffalse t5 goto L_no if category ==
1 goto L_electronics if category
== 2 goto L_books if category ==
3 goto L_clothing
goto L_other
L_electronics: call recommendElectronics; goto L_end
L_books: call recommendBooks; goto L_end
L_clothing: call recommendClothing; goto
L_end L_other: call recommendGeneral L_end:
L_no: call noRecommendation
```

One-line explanation: The IR decomposes recommendation rules into Boolean tests, branches and procedure calls. **ii)**

Case statements A case statement is useful when one selector such as `category` has several discrete values. The compiler can implement it with chained conditional branches for a small number of cases or a jump table when case values are dense. Each category then transfers directly to its recommendation routine.

One-line explanation: Case statements provide organized multi-way selection and can be optimized into efficient jump-based control flow.

iii) Compiler design in e-commerce Efficient lexical/syntax processing, semantic analysis, intermediate-code generation and optimization reduce the cost of processing large numbers of recommendations. Optimized IR can support constant propagation, branch simplification, common-subexpression elimination and efficient calls. A well-designed compiler also improves maintainability because the rule language can remain high-level while the generated code is efficient.

One-line explanation: Good compiler design improves throughput, response time, scalability and maintainability in large e-commerce workloads.

4. Hospital Patient Monitoring System

i) Intermediate code For emergency monitoring:

```
t1 = heartRate > HR_MAX
t2 = heartRate < HR_MIN
t3 = t1 OR t2 t4 =
oxygen < O2_MIN
t5 = t3 OR t4 iffalse t5 goto
L_normal call
triggerEmergencyAlarm
```



```
alert = 1 goto
L_end L_normal:
alert = 0 L_end:
```

This can be extended with temperature and blood-pressure checks using additional temporaries and branches.

One-line explanation: Three-address code converts sensor conditions into simple tests and explicit emergency branches.

ii) Backpatching When the compiler generates `ifFalse t5 goto ?`, the destination may not yet have a final label.

The instruction is placed in a false-list or true-list as appropriate. When the target block such as `L\_normal` or `L\_alarm` is created, backpatching fills the unresolved address. Correct patching is essential because an emergency branch must reach the alarm routine without delay or accidental fall-through.

One-line explanation: Backpatching correctly resolves forward jumps so emergency control flow reaches the intended alert or normal-processing block.

iii) Compiler optimization and reliability Optimization can reduce the number of instructions executed for sensor checks and alarm decisions. Common-subexpression elimination avoids repeating identical tests, branch optimization reduces unnecessary jumps, and efficient register allocation lowers memory traffic. For healthcare monitoring, optimization must never change the meaning of safety conditions; correctness and predictable response are more important than aggressive transformations.

One-line explanation: Careful optimization improves response time while preserving deterministic and semantically correct emergency behavior.

5. Online Airline Reservation System

i) Intermediate code Example: t1 = checkSeat(flight, seat) ifFalse t1 goto L_unavailable t2 = calculateFare(flight, class) t3 = processPayment(card, t2) ifFalse t3 goto L_paymentFail

seatStatus[seat] = RESERVED

ticket = createTicket(passenger, flight, seat)

call confirmBooking(ticket)

goto L_end

L_unavailable: call reportNoSeat

goto L_end

L_paymentFail: call rollbackPayment

L_end:

One-line explanation: The intermediate code separates seat checking, fare calculation, payment, reservation and confirmation into simple operations.

ii) Procedure calls Procedures such as `checkSeat`, `calculateFare`, `processPayment`, `createTicket`, `confirmBooking`, and `rollbackPayment` isolate different reservation responsibilities. Each module can be tested and optimized independently, and the same routines can be reused for booking, modification and cancellation workflows.

One-line explanation: Procedure calls support modular reservation management by encapsulating reusable business operations.

iii) Intermediate languages An intermediate language allows the reservation front end to remain independent of the hardware or execution platform. The same IR can be optimized and then translated into different target architectures, virtual machines or service environments. It also supports platform-neutral verification and common optimization passes for high-volume booking operations.

One-line explanation: Intermediate languages improve portability because the same high-level compiler front end can target multiple execution platforms.

Set 2

1. Industrial Robot Automation System

i) **Intermediate code** t1 = armPosition == target
t2 = obstacleDetected == 0 t3 = t1 AND t2
ifFalse t3 goto L_stop call moveArm call
verifyAssembly goto L_done
L_stop: call emergencyStop
L_done:

One-line explanation: The code makes movement and safety checks explicit before the robot action is executed. ii) **Procedure calls** Functions such as 'moveArm', 'verifyAssembly', 'checkSensor', and 'emergencyStop' isolate robot-control tasks and can be reused by different assembly sequences.

One-line explanation: Modular procedures reduce duplicated control logic and simplify testing and synchronization.

iii) **Compiler optimization** Optimization can reduce instruction count, eliminate redundant sensor checks, improve register use, and streamline procedure calls; deterministic execution is essential for synchronized robotic motion.

One-line explanation: Faster optimized code improves production throughput while helping maintain predictable robot timing.

2. Cybersecurity Intrusion Detection System

i) **Intermediate representation** t1 = packetRate > LIMIT
t2 = failedLogins > LOGIN_LIMIT t3 = suspiciousPort ==
1 t4 = t1 OR t2 t5 = t4 OR t3 ifFalse t5 goto L_normal
call raiseThreatAlert
goto L_end
L_normal: call continueMonitoring
L_end:

One-line explanation: Boolean tests become temporaries and conditional branches that can be optimized independently. ii) **Case statements** A case statement can classify an attack type such as malware, brute force, port scanning, or denial-of-service; the compiler can use branches or a jump table depending on the case distribution.

One-line explanation: Case selection provides clear and efficient multi-category threat classification.

iii) **Optimization** Dead-code elimination, branch optimization, common-subexpression elimination and efficient memory access can reduce monitoring latency, while correctness must be preserved for every threat rule.

One-line explanation: Optimization reduces detection delay, which is critical for real-time cybersecurity response.

3. Digital Payment Gateway System

i) **Intermediate code** t1 = verifyCard(card)
t2 = checkBalance(account, amount)
t3 = t1 AND t2 ifFalse t3 goto
L_reject t4 =
fraudScore(transaction) if t4 >
RISK_LIMIT goto L_fraud call
debit(account, amount) call
confirmPayment goto L_end
L_fraud: call holdTransaction
goto L_end
L_reject: call rejectPayment
L_end:

One-line explanation: The IR explicitly represents verification, fraud checking, payment and rejection paths. ii) **Backpatching** Conditional jumps for verification, fraud detection and rollback may initially contain unknown targets; backpatching records these jumps and fills their destinations when the corresponding labels are generated.

One-line explanation: Backpatching makes forward conditional control flow easy to generate and later resolve.

iii) **Compiler design** A secure compiler should generate correct IR, preserve validation conditions, optimize without changing transaction semantics, and support scalable execution through efficient calls and control flow.

One-line explanation: Good compiler design improves fintech security, reliability, throughput and scalability.

4. Smart Agriculture Monitoring System

i) **Intermediate code** t1 = soilMoisture < MIN_MOISTURE
t2 = temperature > MAX_TEMP
t3 = t1 OR t2 ifFalse t3
goto L_monitor call

startIrrigation goto

L_end

L_monitor: call continueMonitoring

L_end:

One-line explanation: The IR converts environmental conditions into simple Boolean tests and irrigation-control branches. **ii) Boolean expressions** Boolean expressions combine sensor readings, for example `soilMoisture < limit AND temperature > threshold`, to determine whether irrigation or another control action should occur.

One-line explanation: Boolean logic lets the controller make automatic decisions from multiple environmental conditions. **iii) Optimization** Removing redundant tests, simplifying Boolean expressions, reducing branches and improving register/memory use can shorten controller response time and reduce energy consumption.

One-line explanation: Optimization improves timely irrigation decisions and efficient embedded operation.

5. Cloud-Based Video Streaming Platform

i) Intermediate code if bandwidth >= HIGH goto L_high

if bandwidth >= MEDIUM goto L_medium

goto L_low

L_high: quality = HIGH_Q; call allocateHighBandwidth; goto L_end

L_medium: quality = MEDIUM_Q; call allocateMediumBandwidth; goto L_end

L_low: quality = LOW_Q; call allocateLowBandwidth

L_end:

One-line explanation: The intermediate code maps network conditions to quality and resource-allocation actions. **ii)**

Case statements A case on a quality level such as HIGH, MEDIUM, LOW can directly select the corresponding encoding, bitrate and bandwidth routine; a jump table may be efficient for dense selector values.

One-line explanation: Case statements organize multi-level quality management and can reduce complicated conditional chains.

iii) Cloud optimization Optimization reduces CPU work, memory traffic and branching in high-volume streaming paths, helping the system serve more users and adapt quality faster.

One-line explanation: Compiler optimization improves scalability, resource utilization and streaming responsiveness.

Set 3

1. Autonomous Drone Delivery System

i) Intermediate code t1 = obstacleDistance < SAFE_DISTANCE

t2 = battery < LOW_BATTERY t3 = t1 OR t2 if t1 goto L_avoid if

t2 goto L_land call continueRoute goto L_end

L_avoid: call adjustRoute; goto L_end

L_land: call emergencyLanding

L_end:

One-line explanation: The IR explicitly separates obstacle avoidance, battery monitoring and normal navigation. **ii) Boolean expressions** Conditions such as `obstacleDetected AND distanceUnsafe` or `batteryLow OR emergencyDetected` allow the drone to select safe actions automatically.

One-line explanation: Boolean expressions provide the logical decision rules needed for autonomous navigation and safety. **iii) Optimization** Fast branches, efficient procedure calls, reduced memory access and elimination of redundant calculations reduce route-adjustment latency while preserving safety logic.

One-line explanation: Optimization is important because real-time drone decisions must be fast and predictable.

2. Railway Reservation and Scheduling System

i) Intermediate code t1 = checkSeat(train, seat)

ifFalse t1 goto L_full t2 = checkSchedule(train,

time) ifFalse t2 goto L_invalid call reserveSeat

call updateSchedule

goto L_end

L_full: call reportFull

goto L_end

L_invalid: call reportInvalidSchedule

L_end:

One-line explanation: The IR breaks reservation validation and scheduling updates into explicit operations and branches. **ii) Case statements** A case on passenger category can select adult, child, senior, student or other fare/status processing routines.

One-line explanation: Case statements efficiently organize multiple passenger categories and associated rules. **iii)**

Optimization Optimized IR reduces repeated checks, improves database/service calls, and simplifies branches, helping reservation systems process high booking volumes.

One-line explanation: Optimization improves confirmation speed, scalability and transaction efficiency in transportation systems.

3. Intelligent Energy Management System

i) **Intermediate code** t1 = load > MAX_LOAD

t2 = faultDetected == 1

t3 = t1 OR t2 ifFalse t3

goto L_normal if t3 goto

L_fault call

redistributePower

goto L_end

L_fault: call isolateFault

call recoverPower

goto L_end

L_normal: call continueBalance

L_end:

One-line explanation: The code expresses load balancing and fault recovery through explicit tests and labels. ii) **Backpatching** Unresolved jumps from load/fault conditions can be stored in true/false lists and patched when the target recovery or normal-processing labels are known.

One-line explanation: Backpatching helps construct correct forward control flow for fault recovery and redistribution. iii) **Smart-grid optimization** Branch simplification, efficient sensor access, common-subexpression elimination and fast recovery paths reduce controller latency and improve resource use.

One-line explanation: Optimization improves response time and reliability during high-demand grid conditions.

4. AI-Based Fraud Detection System

i) **Intermediate code** t1 = transactionAmount > LIMIT

t2 = unusualLocation == 1 t3 = failedAttempts >

LIMIT2 t4 = t1 OR t2 t5 = t4 OR t3 ifFalse t5 goto

L_normal call generateFraudAlert

goto L_end

L_normal: call approveOrMonitor

L_end:

One-line explanation: The IR converts fraud rules into reusable Boolean tests and alert-control branches.

ii) **Procedure calls** Procedures such as `extractFeatures`, `calculateRisk`, `generateFraudAlert`, and `approveOrMonitor` separate analysis stages and can be reused for different transaction flows.

One-line explanation: Modular procedures make complex fraud detection easier to maintain, test and scale. iii)

Secure financial applications Compiler design must preserve security checks, generate correct control flow, optimize safely, and provide predictable execution for large transaction volumes.

One-line explanation: A reliable compiler reduces semantic and execution errors while supporting scalable secure financial processing.

5. Smart Warehouse Inventory Management System

i) **Intermediate code** t1 = stock < reorderLevel

if t1 goto L_reorder t2 = quantityReceived > 0

if t2 goto L_update goto L_end

L_reorder: call createReorder

goto L_end

L_update: stock = stock + quantityReceived

call updateInventory

L_end:

One-line explanation: The IR represents stock tests, updates and reorder actions as explicit control-flow operations. ii)

Case statements A case on product category can select separate handling for electronics, clothing, food, fragile goods or other categories.

One-line explanation: Case statements provide clear multi-way selection for category-specific inventory rules. iii) **Optimization** Efficient branches, reduced redundant database operations, improved procedure calls and optimized arithmetic can speed stock synchronization.

One-line explanation: Optimization helps maintain inventory accuracy and faster order fulfillment.

Set 4

1. Smart Home Automation System

i) **Intermediate code** t1 = motionDetected == 1
t2 = dark == 1 t3 = t1 AND t2 ifFalse t3 goto
L_lightOff call lightOn
goto L_temp
L_lightOff: call lightOff
L_temp: if temperature > TEMP_LIMIT goto
L_cool call heaterOrIdle goto L_end
L_cool: call airConditioner
L_end:

One-line explanation: The IR turns sensor conditions into simple Boolean tests and device-control branches. **ii) Boolean expressions** Expressions such as `motionDetected AND dark` or `temperature > limit OR smokeDetected` combine sensor states to control devices automatically.

One-line explanation: Boolean logic converts multiple sensor inputs into clear automated decisions. **iii) Optimization** Removing redundant sensor reads, simplifying conditions and reducing branches can improve response time and lower controller workload.

One-line explanation: Optimization makes smart-home responses faster while using embedded resources efficiently.

2. Online Examination Management System

i) **Intermediate code** t1 = verifyStudent(id, password)
ifFalse t1 goto L_authFail t2 = examCategory if t2 ==
1 goto L_online
if t2 == 2 goto L_practical
goto L_standard
L_online: call loadOnlineExam; goto L_eval
L_practical: call loadPracticalExam; goto L_eval
L_standard: call loadStandardExam
L_eval: call evaluateAnswers
call storeResult
goto L_end
L_authFail: call rejectLogin
L_end:

One-line explanation: The IR separates authentication, category selection, exam loading and evaluation into simple control-flow steps.

ii) Case statements A case statement can select different exam categories such as regular, practical, online or special examinations and invoke the corresponding processing routine.

One-line explanation: Case statements simplify multi-category examination processing and improve code organization. **iii) Optimization** Branch optimization, efficient procedure calls, constant propagation and reduced memory access can improve question loading and score processing under heavy load.

One-line explanation: Optimization improves scalability and responsiveness during large online examinations.

3. Intelligent Weather Forecasting System

i) **Intermediate code** t1 = temperature < TEMP_MIN
t2 = rainfall > RAIN_LIMIT t3 =
windSpeed > WIND_LIMIT t4
= t1 OR t2 t5 = t4 OR t3
ifFalse t5 goto L_normal call
generateWeatherAlert
goto L_end
L_normal: call updateForecast
L_end:

One-line explanation: The IR explicitly represents weather thresholds and the emergency-alert branch. **ii) Backpatching** Forward jumps to alert or normal-processing blocks can be generated with unknown targets and later filled using backpatching when the labels are created.

One-line explanation: Backpatching ensures that emergency conditions transfer control to the correct alert routine. **iii) Optimization** Optimizing repeated threshold calculations and branches reduces response latency, while preserving all safety-related conditions exactly.

One-line explanation: Optimization helps real-time forecasting systems generate warnings promptly and efficiently.

4. Automated Library Management System

i) Intermediate code t1 = verifyMember(memberId)

ifFalse t1 goto L_reject t2 =

checkBookAvailable(bookId) ifFalse t2 goto

L_unavailable call issueBook call updateCatalog

goto L_end

L_unavailable: call reportUnavailable

goto L_end

L_reject: call rejectMember

L_end:

One-line explanation: The code separates member verification, availability checking and book issue/update operations. **ii) Procedure calls** Procedures such as `verifyMember`, `issueBook`, `returnBook`, `calculateFine`, and `updateCatalog` isolate library functions and allow reuse across different transactions.

One-line explanation: Procedure calls improve modularity by separating independent library services. **iii) Optimization** Reducing redundant catalog accesses, simplifying branches and improving procedure-call overhead can speed issue, return and fine calculations.

One-line explanation: Optimization improves response time and reliability of educational information services.

5. Industrial Water Quality Monitoring System

i) Intermediate code t1 = pH < PH_MIN

t2 = pH > PH_MAX

t3 = contamination > CONT_LIMIT

t4 = t1 OR t2 t5 = t4

OR t3 ifFalse t5 goto

L_safe call

triggerWarning

if t3 goto L_critical

goto L_end

L_critical: call emergencyShutdown

goto L_end

L_safe: call continueMonitoring

L_end:

One-line explanation: The IR converts water-quality limits into explicit warning and critical-action branches.

ii) Case statements A case statement can classify conditions such as normal, warning, critical contamination, high pH or low pH and select the corresponding response.

One-line explanation: Case selection gives an organized and efficient way to handle multiple water-quality states. **iii)**

Optimization Fast comparisons, branch simplification and efficient sensor/data access reduce alert latency without changing safety semantics.

One-line explanation: Optimization is important because delayed contamination warnings can increase environmental and operational risk.

Set 5

1. Smart Parking Management System

i) **Intermediate code** t1 = slotAvailable == 1
ifFalse t1 goto L_full slot
= allocateSlot(vehicle)
t2 = vehicleType if t2
== 1 goto L_car if t2
== 2 goto L_bike
goto L_other
L_car: fee = hours * CAR_RATE; goto L_pay
L_bike: fee = hours * BIKE_RATE; goto L_pay
L_other: fee = hours *
OTHER_RATE L_pay: call
processPayment(fee) call
confirmParking
goto L_end
L_full: call reportFull
L_end:

One-line explanation: The IR combines slot checking, case-based fee calculation, payment and confirmation.

ii) **Boolean expressions and case statements** Boolean expressions handle conditions such as slot availability and security verification, while case statements select fee rules for different vehicle types or parking categories.

One-line explanation: Together they provide efficient conditional and multi-way control for parking operations. **iii) Optimization** Branch reduction, constant folding of fixed rates, efficient arithmetic and optimized calls can reduce slot-allocation and payment latency.

One-line explanation: Optimization improves real-time response and throughput during peak parking demand.

2. Digital Healthcare Appointment System

i) **Intermediate code** t1 = verifyPatient(patientId)
ifFalse t1 goto L_fail
t2 = checkDoctorAvailability(doctor, time)
ifFalse t2 goto L_unavailable call
bookAppointment call
updatePatientRecord
goto L_end
L_unavailable: call suggestAlternative
goto L_end
L_fail: call rejectPatient
L_end:

One-line explanation: The IR makes patient verification, availability checking and appointment operations explicit. **ii) Procedure calls** Functions such as `verifyPatient`, `bookAppointment`, `cancelAppointment`, `updateRecord`, and `generateReport` separate healthcare services and can be reused across workflows.

One-line explanation: Procedure calls improve modularity and make healthcare operations easier to maintain and test. **iii) Optimization** Efficient database/service calls, branch optimization and reduced redundant calculations can lower confirmation and report-generation time.

One-line explanation: Optimization improves responsiveness and resource use in high-traffic healthcare systems.

3. Automated Toll Collection System

i) **Intermediate code** t1 = identifyVehicle(vehicle)
t2 = validTag(tag) t3 = t1 AND t2 ifFalse t3 goto
L_reject
t4 = vehicleClass toll
= calculateToll(t4)
t5 = processPayment(account,
toll) ifFalse t5 goto L_paymentFail
call openGate
goto L_end
L_reject: call rejectVehicle
goto L_end

L_paymentFail: call paymentError

L_end:

One-line explanation: The IR separates identification, validation, toll calculation, payment and gate control. **ii) Backpatching** Forward branches for invalid vehicles and payment failures can be generated first and patched when their target labels are determined.

One-line explanation: Backpatching resolves conditional control flow cleanly in toll-processing logic. **iii) Optimization** Optimized classification, reduced repeated checks, efficient payment calls and branch simplification improve throughput at high-volume toll points.

One-line explanation: Optimization reduces vehicle-processing latency and helps prevent traffic buildup.

4. Intelligent Fire Detection and Alarm System

i) Intermediate code t1 = smoke > SMOKE_LIMIT

t2 = temperature > TEMP_LIMIT

t3 = t1 OR t2 ifFalse t3

goto L_monitor if t1 AND

t2 goto L_critical call

triggerAlarm

goto L_end

L_critical: call triggerAlarm

call emergencyProtocol

goto L_end

L_monitor: call continueMonitoring

L_end:

One-line explanation: The IR explicitly represents normal monitoring, alarm and critical emergency branches. **ii) Case statements** A case statement can classify conditions such as normal, smoke warning, high temperature, both sensors critical, or sensor fault and select the appropriate emergency response.

One-line explanation: Case statements organize multiple emergency states and their corresponding actions. **iii) Optimization** Fast sensor comparisons, reduced branches and efficient alarm procedure calls can shorten response time while preserving all safety conditions.

One-line explanation: Optimization is valuable because faster correct alarm generation can improve industrial safety response.

5. AI-Based Customer Support Chatbot

i) Intermediate code t1 = classifyQuery(input)

if t1 == 1 goto L_account if t1 == 2 goto

L_payment if t1 == 3 goto L_product goto

L_general

L_account: call handleAccountQuery; goto L_end

L_payment: call handlePaymentQuery; goto L_end

L_product: call handleProductQuery; goto L_end

L_general: call handleGeneralQuery

L_end: response = generateResponse()

send(response)

One-line explanation: The IR classifies the query and calls a specialized handler before generating the final response. **ii) Procedure calls** Separate procedures such as `classifyQuery`, `handleAccountQuery`, `handlePaymentQuery`, and `generateResponse` make each chatbot capability independently reusable and testable.

One-line explanation: Procedure calls support modular AI workflows and simplify maintenance and extension. **iii) Optimization** Branch optimization, efficient string/data handling, common-subexpression elimination and reduced call overhead can lower response latency.

One-line explanation: Compiler optimization improves chatbot throughput, response time and scalability during large user loads.

Key revision: Three-address code uses temporaries and labels; backpatching fills unresolved jump targets; case statements implement multi-way selection; procedure calls provide modularity; intermediate languages provide machine independence; optimization reduces execution cost while preserving program meaning.